

Trainer ছাড়া “scratch” থেকে **training loop** লিখতে যাবে

1) Dataset লোড করা (Part -1)

```
from datasets import load_dataset
```

```
raw_datasets = load_dataset("glue", "mrpc")
```

কী হচ্ছে?

- `load_dataset("glue", "mrpc")` → GLUE benchmark-এর MRPC task ডাউনলোড/লোড করে।
- `raw_datasets` সাধারণত একটা `DatasetDict`:
 - `raw_datasets["train"]`
 - `raw_datasets["validation"]`
 - (কখনও `test`ও থাকে)

MRPC কী?

Microsoft Research Paraphrase Corpus: দুইটা বাক্য একই অর্থ বহন করে কিনা (paraphrase কিনা) → binary classification।

2) Model checkpoint + tokenizer লোড

```
from transformers import AutoTokenizer
```

```
checkpoint = "bert-base-uncased"
```

```
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
```

কী হচ্ছে?

- `checkpoint` = কোন pretrained model family ব্যবহার করবে (এখানে BERT base uncased)
- `AutoTokenizer.from_pretrained(...)` = ওই মডেলের জন্য matching tokenizer লোড করে

Tokenizer কাজ: text → token ids (`input_ids`) + দরকারি mask/type ids তৈরি করা।

3) tokenize_function লেখা

```
def tokenize_function(example):  
    return tokenizer(example["sentence1"], example["sentence2"], truncation=True)
```

এখানে **input example** কী?

`raw_datasets` থেকে একেকটা row/example আসবে, যার মধ্যে থাকে:

- `example["sentence1"]`
- `example["sentence2"]`
- `example["label"]`
- `example["idx"]`

tokenizer কে দুইটা **sentence** দেওয়া কেন?

MRPC হলো sentence pair task, তাই tokenizer-কে 2টা বাক্য দিলে সে:

- দুইটা বাক্য জোড়া লাগিয়ে encode করে
- `token_type_ids` ব্যবহার করে sentence A/B আলাদা করে
- `attention_mask` বানায়

truncation=True কেন?

BERT-এর max length fixed (সাধারণত 512)।

দুইটা sentence মিলিয়ে যদি length বেশি হয়ে যায়, তাহলে কেটে max limit-এর মধ্যে আনে। না হলে error/overflow হতে পারে।

4) Dataset tokenize করা (map)

```
tokenized_datasets = raw_datasets.map(tokenize_function, batched=True)
```

map কী করছে?

Dataset-এর প্রতিটা example এর উপর `tokenize_function` চালিয়ে নতুন কলাম যোগ করছে:

- `input_ids`
- `attention_mask`
- `token_type_ids` (BERT হলে)

batched=True মানে?

একটা একটা row না পাঠিয়ে, একসাথে অনেকগুলো row পাঠায় (batch আকারে) → দ্রুত কাজ হয়।
এখানে **example** আসলে batch-dict হয়ে যায়, tokenizer এটাও হ্যান্ডেল করতে পারে।

5) Data collator (Dynamic padding)

```
from transformers import DataCollatorWithPadding
```

```
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

এটা কেন দরকার?

একেকটা sentence pair tokenize করলে length ভিন্ন হয়।

DataLoader যখন batch বানাবে, সব sample-এর length equal করতে হবে।

DataCollatorWithPadding:

- batch-এর মধ্যে সর্বোচ্চ length বের করে
- বাকি গুলোতে padding token যোগ করে
- tensors stack করে ফেরত দেয়

ফলে: memory-efficient padding (পুরো dataset max-length পর্যন্ত padding না দিয়ে, batch-wise padding)।

কী হচ্ছে এখানে **(Part -2)**

`tokenized_datasets` = তোমার tokenized dataset (HuggingFace Datasets object)
এর ভেতরে এখনো এমন কিছু কলাম আছে যেগুলো মডেল ইনপুট হিসেবে নেয় না (যেমন `sentence1`, `sentence2`, `idx`), আর `label` কলামের নামও মডেলের expectation অনুযায়ী ঠিক করা হয়নি।

তাই ৩টা কাজ:

1. অপ্রয়োজনীয় কলাম বাদ
2. `label` নাম বদলে `labels`
3. ডেটা টাইপ list → PyTorch tensor

1) অপ্রয়োজনীয় কলাম বাদ দেওয়া

```
tokenized_datasets = tokenized_datasets.remove_columns(["sentence1", "sentence2", "idx"])
```

কেন দরকার?

- `sentence1`, `sentence2` হলো raw text—মডেল এগুলো চায় না (মডেল চায় token ids)
- `idx` শুধু dataset-এর index/id, training-এ লাগে না
- এগুলো রেখে দিলে DataLoader থেকে batch বানানোর সময় extra field চলে আসবে, আর `model(**batch)` দিলে error বা mismatch হতে পারে

ফলাফল: dataset থেকে ওই ৩টা কলাম চলে যাবে।

2) `label` → `labels` rename করা

```
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
```

কেন দরকার?

Transformers মডেলের forward signature সাধারণত এমন:

- `input_ids=...`
- `attention_mask=...`
- `token_type_ids=...` (কিছু মডেলে থাকে)
- `labels=...` ✓

তুমি যদি `labels` নাম না দাও, তাহলে মডেল label পাবে না, ফলে:

- loss অটো compute হবে না (বা error হতে পারে)

- training loop-এ extra কাজ করতে হবে

ফলাফল: এখন dataset-এর label কলামের নাম হবে `labels`।

3) ডেটাকে **PyTorch tensor** বানানো

```
tokenized_datasets.set_format("torch")
```

কেন দরকার?

HuggingFace dataset ডিফল্টভাবে অনেক সময় Python list/NumPy ফরম্যাটে ফেরত দেয়। কিন্তু PyTorch training loop-এ আমাদের tensor দরকার, কারণ:

- `.to(device)` করতে হবে
- model tensor input চায়
- loss/backprop সব tensor ভিত্তিক

ফলাফল: `tokenized_datasets["train"][0]` টাইপের আউটপুট এখন `torch.Tensor` হবে।

4) কলামগুলো চেক করা

```
tokenized_datasets["train"].column_names
```

Expected output:

```
["attention_mask", "input_ids", "labels", "token_type_ids"]
```

এই কলামগুলো কী?

- **input_ids**: tokenized text-এর সংখ্যাগুলো (wordpiece ids)
- **attention_mask**: কোনটা real token (1) আর কোনটা padding (0)
- **token_type_ids**: sentence pair হলে A/B পার্থক্য বোঝায় (BERT-type মডেলে)
- **labels**: তোমার ground-truth class (0/1)

এগুলোই মডেল “accept” করে, তাই batch দিলে সমস্যা হবে না।

1) **DataLoader** কেন লাগে? (Part -3)

`tokenized_datasets["train"]` হলো dataset (অনেকগুলো example)।

কিন্তু training করার সময় আমরা একেকবার সব ডেটা একসাথে মডেলে দিতে পারি না। তাই ডেটা ভেঙে ছোট ছোট **batch** করে নিতে হয়।

DataLoader কাজ করে:

- dataset থেকে batch বানায়
- (train হলে) shuffle করে
- batching + padding ঠিকভাবে করে (collate_fn দিয়ে)
- iteration সহজ করে: `for batch in train_dataloader: ...`

2) কোডটা কী করছে?

```
from torch.utils.data import DataLoader
```

এটা PyTorch-এর DataLoader ইমপোর্ট।

Train dataloader

```
train_dataloader = DataLoader(  
    tokenized_datasets["train"],  
    shuffle=True,  
    batch_size=8,  
    collate_fn=data_collator  
)
```

এখানে প্রতিটা **argument**:

- `tokenized_datasets["train"]`
→ training split-এর dataset
- `shuffle=True`
→ প্রতিটা epoch-এ ডেটার order এলোমেলো হবে
কেন দরকার?
যদি order একই থাকে, model কিছু pattern ধরে ফেলতে পারে, generalization খারাপ হতে পারে।
- `batch_size=8`
→ একবারে ৮টা sample মডেলে যাবে
(GPU/মোমোরি অনুযায়ী batch size ঠিক করা হয়)
- `collate_fn=data_collator`
→ এটা খুব গুরুত্বপূর্ণ
DataLoader যখন ৮টা sample একসাথে batch বানাবে, তখন প্রত্যেক sample-এর sequence length

আলাদা হতে পারে।

data_collator কাজ:

- একই batch-এর মধ্যে সর্বোচ্চ length বের করে
- বাকিগুলোতে padding যোগ করে
- তারপর সবকিছু tensor আকারে stack করে

Eval dataloader

```
eval_dataloader = DataLoader(  
    tokenized_datasets["validation"],  
    batch_size=8,  
    collate_fn=data_collator  
)
```

এখানে **shuffle=False** (ডিফল্ট) রাখা হয়েছে।

কেন **eval** এ **shuffle** না?

Evaluation-এ order matter না, আর reproducibility/consistent metrics-এর জন্য সাধারণত shuffle করা হয় না।

3) Batch ঠিক আসছে কিনা “চেক” করা

```
for batch in train_dataloader:  
    break
```

এটা training চালায় না। শুধু:

- DataLoader থেকে প্রথম **batch** নিয়ে আসে
- তারপর **break** দিয়ে loop থামিয়ে দেয়

অর্থাৎ “একটা batch দেখাই”—এই উদ্দেশ্য।

4) Batch-এর ভেতরের tensor shape দেখা

```
{k: v.shape for k, v in batch.items()}
```

মানে: batch dictionary-এর প্রতিটা key/value এর shape প্রিন্ট করো।

Output:

```
{  
'attention_mask': torch.Size([8, 65]),  
'input_ids': torch.Size([8, 65]),  
'labels': torch.Size([8]),  
'token_type_ids': torch.Size([8, 65])  
}
```

5) এই **shape**-গুলার মানে কী?

[8, 65] কেন?

- 8 = batch_size (৮টা example একসাথে)
- 65 = ওই batch-এ সর্বোচ্চ token length (max length within that batch)

তাই:

- **input_ids**: 8টা sentence → padding করে 65 length করা হয়েছে
- **attention_mask**: same shape, padding কোথায় আছে সেটা দেখায়
- **token_type_ids**: BERT-style model হলে same shape

labels: [8] কেন?

প্রতিটা sample-এর জন্য ১টা label, তাই মোট ৮টা label → shape [8]

6) কেন **shape** আলাদা হতে পারে?

তুমি যে লাইনটা দিলা:

```
shapes slightly different ... shuffle=True ... padding to max length inside the batch
```

এটার মানে:

- **shuffle=True** হলে প্রতি বার batch-এর ভেতরে অন্য অন্য sample আসবে
- ভিন্ন sample মানে ভিন্ন length
- **data_collator** প্রতি batch-এ আলাদা max length ধরে padding দেয়
- তাই একবার [8, 65], আরেকবার [8, 72]—এটা normal

1) মডেল ইনিশিয়ালাইজ করা (Part - 4)

```
from transformers import AutoModelForSequenceClassification
```

```
model = AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)
```

এখানে কী হচ্ছে?

- `AutoModelForSequenceClassification` = এটা এমন একটা wrapper যা checkpoint দেখে বুঝে নেয় কোন base transformer লাগবে (BERT/Roberta/etc) + উপরে classification head বসায়।
- `.from_pretrained(checkpoint, ...)` = pretrained weights লোড করে (আগে থেকে ট্রেন করা মডেল)।
- `num_labels=2` = তোমার টাস্ক **binary classification** (দুইটা ক্লাস, যেমন 0/1)।

ফলাফল:

`model` এখন এমন একটা network যেটা input sentence → 2টা class score (logits) দিবে।

2) Batch মডেলে pass করা (sanity check)

```
outputs = model(**batch)
print(outputs.loss, outputs.logits.shape)
```

`model(**batch)` মানে কী?

আগে `batch` ছিল dictionary টাইপ, যেমন:

- `batch["input_ids"]`
- `batch["attention_mask"]`
- `batch["token_type_ids"]`
- `batch["labels"]`

`**batch` দিলে Python ওই dictionary-টা unpack করে function arguments বানায়, প্রায় এমন:

```
outputs = model(
    input_ids=batch["input_ids"],
    attention_mask=batch["attention_mask"],
    token_type_ids=batch["token_type_ids"],
    labels=batch["labels"]
)
```

কেন এটা গুরুত্বপূর্ণ?

এখানে তুমি যাচাই করছো:

- dataset columns ঠিক আছে কিনা
- dataloader batch বানাতে পারছে কিনা
- model forward pass করতে পারছে কিনা
- loss compute হচ্ছে কিনা (labels ঠিকভাবে যাচ্ছে কিনা)

3) Output বোঝা: loss এবং logits

Output:

```
tensor(0.5441, grad_fn=<NllLossBackward>) torch.Size([8, 2])
```

outputs.loss কী?

- এটা training-এর loss value
- `labels` দেওয়া আছে বলে model নিজে থেকেই loss calculate করেছে
- `0.5441` মানে: এই batch-এ prediction vs label mismatch-এর পরিমাণ (কম হলে ভালো)

`grad_fn=<...>` মানে:

- এটা computational graph-এর অংশ
- পরে `.backward()` করলে gradient হিসাব করতে পারবে

outputs.logits.shape = [8, 2] কেন?

- `8` = batch_size (৮টা sample)
- `2` = num_labels (দুইটা class)

প্রতিটা sample-এর জন্য model ২টা raw score দেয় (এগুলোই logits):

- class 0 score
- class 1 score

পরে softmax দিলে probability পাওয়া যাবে।

4) এখন কেন **Optimizer** দরকার?

Training loop-এ তুমি loss বের করার পর:

- `loss.backward()` → gradient বের করবে
- optimizer → ওই gradient দিয়ে model weights update করবে

তাই optimizer ছাড়া training হবে না।

5) AdamW optimizer সেট করা

```
from torch.optim import AdamW
```

```
optimizer = AdamW(model.parameters(), lr=5e-5)
```

এখানে কী হচ্ছে?

- `model.parameters()` = model-এর সব trainable weights
- `AdamW` = Adam-এর মতোই, কিন্তু **weight decay** handling আলাদা/ভালো (transformer fine-tuning এ standard)
- `lr=5e-5` = learning rate (Trainer default)

learning rate মানে কী?

প্রতিটা update step-এ weight কতটা পরিবর্তন হবে।

- খুব বেশি হলে training unstable
- খুব কম হলে training slow/underfit

Transformers-এর জন্য `5e-5` একটা common starting point।

1) Scheduler কেন লাগে? (Part -5)

Optimizer (AdamW) শুধু weights update করে। কিন্তু **learning rate** একই রাখলে অনেক সময়:

- শুরুতে দ্রুত শেখা দরকার
- পরে ধীরে ধীরে ছোট ছোট update দরকার (fine-tuning)

তাই Trainer ডিফল্টভাবে ব্যবহার করে:

Linear decay: শুরুতে $lr = 5e-5$, ধীরে ধীরে কমতে কমতে শেষের দিকে 0 হয়ে যাবে।

2) `num_training_steps` কেন দরকার?

Scheduler কে বলতে হয়: “মোট কয়টা update step হবে?”

একটা update step = এক batch শেষ করে optimizer step করা

তাই:

- ১ epoch = `len(train_dataloader)` টা batch
- total steps = `num_epochs * len(train_dataloader)`

3) কোড

```
from transformers import get_scheduler
```

Scheduler বানানোর helper function।

```
num_epochs = 3
```

Trainer ডিফল্ট ৩ epoch চালায়, তাই এখানেও ৩ রাখা হয়েছে।

```
num_training_steps = num_epochs * len(train_dataloader)
```

- `len(train_dataloader)` = এক epoch-এ কয়টা batch আছে
(যেমন `dataset_size / batch_size`)
- সেটাকে ৩ দিয়ে গুণ করলে মোট training steps

উদাহরণ আউটপুট:

1377

মানে: পুরো **training**-এ **optimizer** মোট **1377** বার **update** হবে।

4) Scheduler বানানো

```
lr_scheduler = get_scheduler(  
    "linear",  
    optimizer=optimizer,  
    num_warmup_steps=0,  
    num_training_steps=num_training_steps,  
)
```

এর মানে:

- **"linear"**: learning rate লিনিয়ার ভাবে কমবে
 $5e-5 \rightarrow 0$
- **optimizer=optimizer**: কোন optimizer-এর lr পরিবর্তন করবে সেটা
- **num_warmup_steps=0**: শুরুতে warmup নেই
(warmup থাকলে শুরুতে ধীরে ধীরে lr বাড়ানো হয়, তারপর decay)
- **num_training_steps=...**: মোট কয় স্টেপে 0 তে নামবে

5) বাস্তবে কী হবে?

ধরা যাক:

- initial lr = $5e-5$
- total steps = 1377

তাহলে প্রতিটা step শেষে lr একটু একটু করে কমবে, এবং step 1377 এ গিয়ে প্রায় 0 হবে।

প্রায় প্রতি **step** কমবে $\approx 5e-5 / 1377$

(এটা ধারণা দেওয়ার জন্য; exact schedule internal implementation অনুযায়ী হবে)

1) GPU/CPU device সেট করা (Part -6)

```
import torch
```

```
device = torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")  
model.to(device)  
device
```

এখানে কী হচ্ছে?

- `torch.cuda.is_available()` চেক করে GPU আছে কিনা।
- GPU থাকলে `device="cuda"`, না থাকলে `device="cpu"`।
- `model.to(device)` মানে মডেলের সব parameters/weights GPU (বা CPU) তে পাঠানো।

কেন দরকার?

Model যদি GPU তে থাকে আর data CPU তে থাকে (বা উল্টো), তাহলে error হবে:

Expected all tensors to be on the same device...

Output যদি আসে `device(type='cuda')` → GPU ব্যবহার হচ্ছে।

2) progress bar (tqdm)

```
from tqdm.auto import tqdm
progress_bar = tqdm(range(num_training_steps))
```

কেন?

Training কতদূর হলো সেটা visual ভাবে দেখাতে।

`num_training_steps` = মোট কয়টা batch-step হবে (তুমি আগে হিসাব করেছিলে: `epochs × len(dataloader)`)।

`tqdm(range(...))` → একটা progress bar তৈরি করে, যেটা আমরা প্রতি step শেষে update করবো।

3) Train mode অন করা

```
model.train()
```

এটা কী করে?

মডেলকে training mode-এ দেয়।

এটার প্রভাব পড়ে কিছু layer-এ:

- **Dropout**: training এ ON, eval এ OFF
- **BatchNorm** (যদি থাকে): training behavior আলাদা

সুতরাং **training loop** শুরু করার আগে `model.train()` জরুরি।

4) Training loop: epoch + batch

```
for epoch in range(num_epochs):  
    for batch in train_dataloader:  
        ...
```

ধারণা

- `num_epochs` বার পুরো dataset ঘুরবে
- প্রতিটা epoch-এ dataloader batch-by-batch data দিবে

5) Batch কে device-এ পাঠানো

```
batch = {k: v.to(device) for k, v in batch.items()}
```

এখানে কী হচ্ছে?

`batch` একটা dictionary, যেমন:

- `input_ids`
- `attention_mask`
- `token_type_ids`
- `labels`

প্রতিটা tensor কে `.to(device)` করে GPU/CPU তে পাঠানো হচ্ছে।

কেন দরকার?

Model device-এ আছে, data-ও সেই device-এ থাকতে হবে।

6) Forward pass (prediction + loss)

```
outputs = model(**batch)  
loss = outputs.loss
```

`model(**batch)` কী করে?

batch unpack করে মডেলে দেয়:

- input_ids/attention_mask/token_type_ids → forward
- labels দেওয়া আছে বলে → model নিজেই loss compute করে দেয়

`loss` হলো scalar tensor (একটা সংখ্যা টাইপ), যেটা `backward`-এ লাগবে।

7) Backpropagation (gradient বের করা)

`loss.backward()`

এখানে কী হচ্ছে?

- loss থেকে শুরু করে মডেলের সব weights এর জন্য gradient হিসাব হচ্ছে
- gradients জমা হয় `param.grad` এ

এখনো **weights update** হয় নাই—শুধু gradients তৈরি হলো।

8) Weight update + LR scheduler update + gradient reset

`optimizer.step()`
`lr_scheduler.step()`
`optimizer.zero_grad()`

`optimizer.step()`

- gradient ব্যবহার করে weights আপডেট করে (AdamW algorithm)

`lr_scheduler.step()`

- learning rate schedule অনুযায়ী lr এক ধাপ কমায় (linear decay: $5e-5 \rightarrow 0$)

সাধারণভাবে sequence: `optimizer.step()` তারপর `scheduler.step()` — এইটাই common।

`optimizer.zero_grad()`

- আগের batch-এর gradients মুছে দেয়

কেন মুছতে হয়?

PyTorch default ভাবে gradient **accumulate** করে।

তুমি যদি zero না করো, পরের batch-এর gradient আগেরটার সাথে যোগ হয়ে যাবে (unintended)।

9) progress bar আপডেট

```
progress_bar.update(1)
```

প্রতিটা training step শেষে progress bar এক ধাপ এগোয়।

10) এই loop এ “evaluation/reporting” নেই কেন?

এখানে শুধু training হচ্ছে—loss বের হচ্ছে, weights update হচ্ছে।

কিন্তু model কতটা ভালো করছে (accuracy/F1) সেটা দেখতে হলে:

- `model.eval()` দিয়ে evaluation mode
- `torch.no_grad()` দিয়ে gradient বন্ধ
- `eval_dataloader` দিয়ে predictions
- metrics compute

তাই শেষে বলছে: **evaluation loop** যোগ করতে হবে।